High Speed 64 Bit Vedic & Booth Multiplier Implementation Using FPGA

Poorvika Singh
Department of Electronics Engineering
J.C. Bose University of Science and Technology, YMCA
Faridabad, India
poorvikasingh24@gmail.com

Rohan Bansal
Department of Electronics Engineering
J.C. Bose University of Science and Technology, YMCA
Faridabad, India
rohan8686sc @gmail.com

Nitin Sachdeva
Department of Electronics Engineering
J.C. Bose University of Science and Technology, YMCA
Faridabad, India
nitinsachdeva@jcboseust.ac.in

Pradeep Dimri
Department of Electronics Engineering
J.C. Bose University of Science and Technology, YMCA
Faridabad, India
pkdimri@gmail.com

2024 3rd International Conference on Artificial Intelligence For Internet of Things (AIIoT) | 979-8-3503-7212-0/24/$31.00 ©2024 IEEE | DOI: 10.1109/AIIoT58432.2024.10574654

***Abstract*—This work aims to showcase the significance of Vedic and Booth mathematics in the realm of Computer Applications. Good precision, speed, area reduction, and low power consumption are primary features of a multiplier. Multiplication operations require speed and speed can be enhanced by decreasing the number of steps needed to complete the computation. The requirement for high-speed operations in digital devices has become a primary need nowadays. This research delves into two distinct multiplier algorithms—namely, the Vedic Multiplication Algorithm and the Booth Multiplication Algorithm—conducting an extensive exploration to determine through comparison which multiplier demonstrates superior performance across various metrics such as delay, power, and other commonly utilized performance indicators. While the Booth Algorithm involves a reduction in partial products, the Vedic Multiplication Algorithm is built on Urdhva Triyakbhyam sutra of Vedic Mathematics. The complete design, simulation, and synthesis procedures were executed through Xilinx Vivado, utilizing Verilog as the Hardware Description Language (HDL). The hardware implementation was conducted on an FPGA (Field Programmable Gate Array).**

Keywords—HDL, UrdhvaTriyakbhyam, Xilinx, Vivado, FPGA

I. INTRODUCTION

This article explores the design and implementation of different multipliers. Multipliers remain one of the most crucial as well as critical components of computation. They hold a prominent place in DSP (Digital Signal Processing) and microprocessors. They are widely used in MAC (Multiplication and Accumulation) units of processors and multimedia applications. Multiplication is an essential function used in digital filters [1]. In the present world, where speed, power and area are ever ameliorating, there is a need to enhance the performances of the multipliers to contribute to these indices. The performance parameters concentrated on here are path delay, power consumption and logic block utilization by the multiplier.

Multipliers require more hardware resources and have more latency[2][3]. The most elementary layout of the multiplier designed and used is the Array Multiplier. While it is easy to design and implement for small numbers, for two 64-bit numbers it becomes tedious and cumbersome. The sums and carries get carried forward, resulting in high latency. The worst case delay for Array multiplier is of (2N+1)tpd [3][4]. Nowadays, the Internet can be accessed almost anywhere, due to which the border between real and virtual worlds gets dimmer from time to time. Most people in the world now believe this virtual world to be real, considering information security just as important as physical security in the actual world.

Many VLSI implementable algorithms have been developed for fast multiplication. This project focuses on studying two of these algorithms namely, Vedic Multiplication Algorithm and Booth Algorithm. Both algorithms are used to perform high-speed multiplications very efficiently [5].

Vedic Multiplication Algorithm [6] owes its origin to the book “Vedic Mathematics” by Indian monastic and mathematician Shri Bharati Krishna Tirtha. The book shares the concept of Vedic Mathematics which gives us novel and innovative ways to solve difficult mathematical problems. Vedic Mathematics consists of sixteen fundamental principles called sutras and thirteen corollaries known as sub-sutras, which greatly facilitate our calculations. The Vedic Multiplication Algorithm relies on the Urdhva Triyakbhyam Sutra from Vedic Mathematics, which is particularly crucial when multiplying two substantial numbers [7][8]. Vedic multiplication methods are not only fascinating historically but also offer alternative approaches to conventional multiplication algorithms, providing insights into computational efficiency and mental math strategies.

A.D. Booth introduced Booth Multiplication in 1951, specifically designed for the multiplication of two signed numbers. This method executes signed multiplication using the 2’s complement form, where the Most Significant Bit (MSB) signifies the sign bit. It minimizes the quantity of generated partial products [9]. While both the Booth Algorithm and the Vedic Algorithm are methods utilized for multiplication, they exist within separate domains and possess unique characteristics [10]. The Booth multiplier is a hardware-specific technique used for efficient binary multiplication in digital systems, while Vedic multiplication techniques are mental arithmetic strategies used for quick multiplication calculations without involving physical hardware. The two serve different purposes in different contexts.

An FPGA, which stands for "Field-Programmable Gate Array," represents a type of integrated circuit (IC) that users can modify even after its fabrication. FPGAs are excellent for prototyping as they can be reconfigured and customized by the user to carry out a variety of jobs and operations, in contrast to conventional application-specific integrated circuits (ASICs), which are built for a single purpose and cannot be modified once manufactured.

Overall, FPGAs are versatile hardware platforms that provide the flexibility to create custom digital circuits, accelerating certain types of computations and enabling the implementation of specialized hardware functionality.

II. Literature Review

A comparative analysis of the delay parameter has been performed for the 16-bit Array multiplier, Booth as well as the Vedic multiplier [3]. When two numbers, m and n are multiplied the output can be +mr, 2r-mr, 2m-mr and 4-2m-2r+mr depending on the signs of the numbers. A correction needs to be applied to all results except for first where both numbers are positive. The corrections need examination and thus, are deemed undesirable. Booth's Algorithm was developed to perform multiplication in a uniform manner without the need to examine the signs of the interacting numbers hence eliminating the need for corrections [11]. A Booth Multiplier incorporating a Booth Decoder was suggested. The primary role of the Booth Decoder is to translate the provided input into a corresponding Booth value. With an increased number of zeroes present, the circuit experiences a reduction in power consumption [12]. Among several sutras in Vedic Mathematics, the universal formula of multiplication is known as Urdhva Triyakbhyam Sutra. This sutra holds its applicability true even for the multiplication of digital bits [13][14]. The diagram illustrating a high-speed 16 x 16-bit Vedic Multiplier is presented and explained [15][16].

III. Booth Multiplier Algoritm

A Booth multiplier is a hardware technique used in digital circuits, particularly in digital signal processing and arithmetic operations, to efficiently perform binary multiplication. It reduces the number of partial product additions required in a traditional binary multiplication process, thereby speeding up the multiplication operation. The Booth Multiplication Algorithm was devised by Andrew Donald Booth in 1951 which enabled the multiplication of two signed numbers represented in their 2's complement form. To speed up the multiplication algorithm, Booth encoding performs several steps of multiplication at once [17] [18]. Booth's algorithm analyses consecutive pairs of bits within the multiplier to identify sequences of 0s and 1s. It replaces pattern of repeated bits with a sequence of additions and subtractions, effectively minimizing the total number of required operations. The steps for implementing the Booth algorithm are outlined below:

1. The multiplicand resides in the M register, while the multiplier occupies the Q register.

2. Adjacent to the Least Significant Bit (LSB) Q0, a 1-bit register labelled Q-1 is logically positioned.

3. Initially, both A and Q-1 are set to 0.

4. Control logic examines the combination of Q0 and Q-1 (Q0Q-1). If Q0Q-1 equals 00 or 11, all bits in A, Q, and Q-1 undergo an arithmetic right shift by 1 bit. When Q0Q-1 equals 10, the outcome of (A–M) is stored in A, followed by an arithmetic right shift of all bits in A, Q, and Q-1 by 1 bit. Similarly, if Q0Q-1 equals 01, the result of (A+M) is held in A, succeeded by an arithmetic right shift of all bits in A, Q, and Q-1 by 1 bit. The arithmetic shift operation retains the MSB (AN-1) in AN-2 while also preserving it in AN-1, ensuring the sign of the number in the concatenation of A and Q, denoted as AQ.

5. The outcome of the multiplication emerges within the concatenation of A and Q, denoted as AQ [20].

Here, the multiplication is dependent on the clock. This algorithm is particularly useful in hardware implementations and digital circuits, where it optimizes the multiplication process by cutting down on the number of basic arithmetic operations required, thereby enhancing computational efficiency. Booth's algorithm forms the foundation for various hardware-based multiplication units in processors and is essential for speeding up multiplication operations in computer architectures. The algorithm is illustrated in Fig. 1.

The simulation of 64 x 64 bit Booth Multiplier results after simulating the Verilog code in Xilinx Vivado are shown in Fig. 2.

An N-bit Booth Multiplier has been designed using a clock, control signals and ASR (Arithmetic Shift Right) operation [19]. The schematic for the 64 x 64-Bit Booth Multiplier is generated utilizing LUTs and multiplexers, as depicted in Fig. 3.

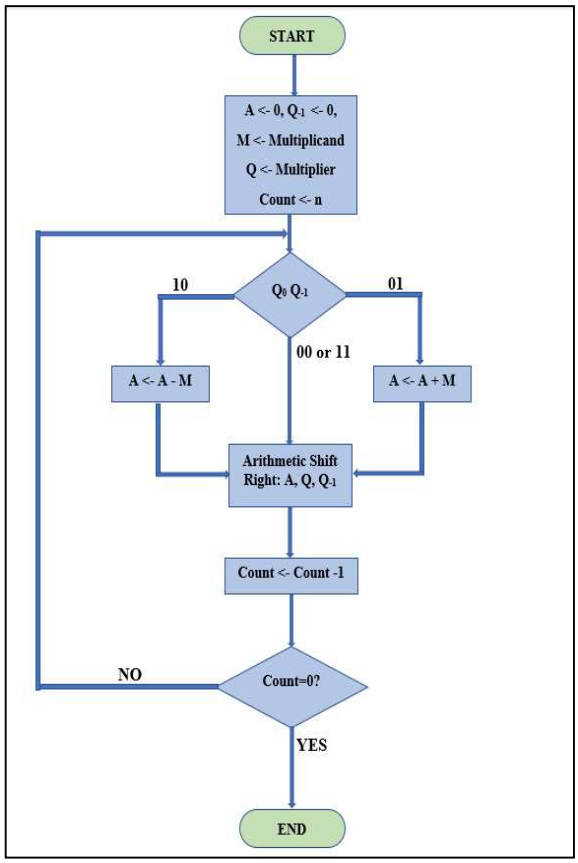

Fig. 1. Booth Multiplication Algorithm.

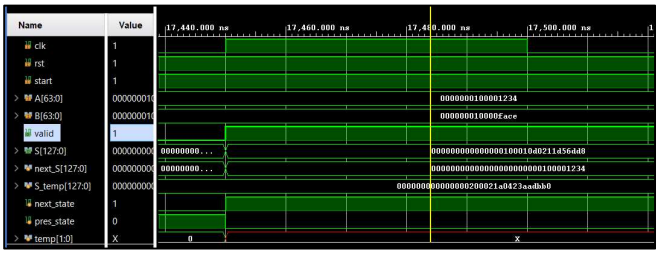

Fig. 2. Simulation results of 64 x 64 bit Booth Multiplier.

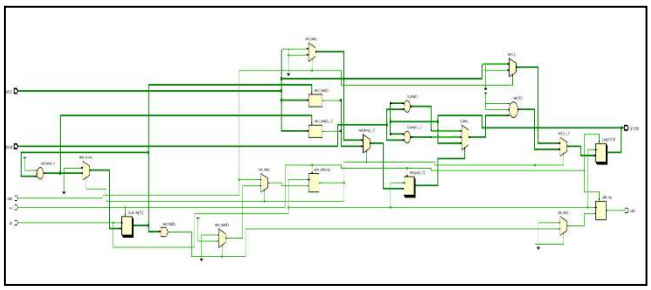

Fig. 3. Schematic of 64 x 64 bit Booth Multiplier.

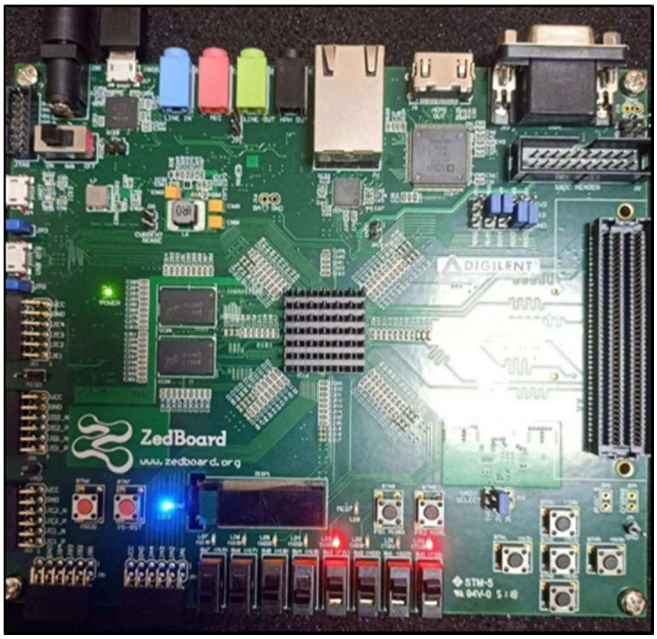

Fig. 4. FPGA Implementation of Booth Multiplier

We used an FPGA synthesis tool called Xilinx Vivado to synthesize the RTL code. The synthesis tool translates the RTL code into a netlist at the gate level, aligning it with the resources available in the particular FPGA architecture.

Once the design is synthesized, placed, and routed, the synthesis tool generates a configuration file (bitstream or programming file) that contains instructions for programming the FPGA. Use the generated configuration file to program the FPGA. The bitstream is loaded onto the FPGA, configuring it to behave according to the design described in the RTL code. Thus, the implementation of the design occurs on the ZED Board FPGA, as depicted in the Fig. 4.

IV. Vedic Multiplier Algorithm

A Vedic multiplier, also known as a Vedic multiplication technique or Vedic mathematics, is an ancient Indian method for performing fast mental arithmetic. It's based on a set of mathematical rules and techniques that enable efficient multiplication, division, addition, and subtraction. The Vedic multiplier specifically refers to the technique used to multiply two numbers together. The very word 'Veda' means boundless repository of all the knowledge and is the root word from which the term 'Vedic' has been derived. Vedic Mathematics obtains its vogue from the inexpressible simplicity as well as ease that it offers while being applied to arduous mathematical problems [21].

The Sixteen Sutras of Vedic Mathematics offer out-of-the-box procedures to solve an otherwise laborious mathematical question. The Sutra we are focusing on here is Urdhva Triyakbhyam. 'Urdhva' means 'Vertically' and 'Triyakbhyam' means 'Crosswise' [22]. This Sutra applies universally to all multiplication cases. It achieves the multiplication result through concurrent processing of partial products and additions, thereby reducing the latency of the multiplier [23]. The computation done here is independent of the clock. It enables quicker mental calculations and finds applications in digital signal processing, computer arithmetic, and hardware design, as it offers efficient ways to perform multiplication operations in computer systems. The procedure for multiplying two 4-bit numbers using the Urdhva Triyakbhyam Sutra is outlined in the diagram provided as Fig. 5.

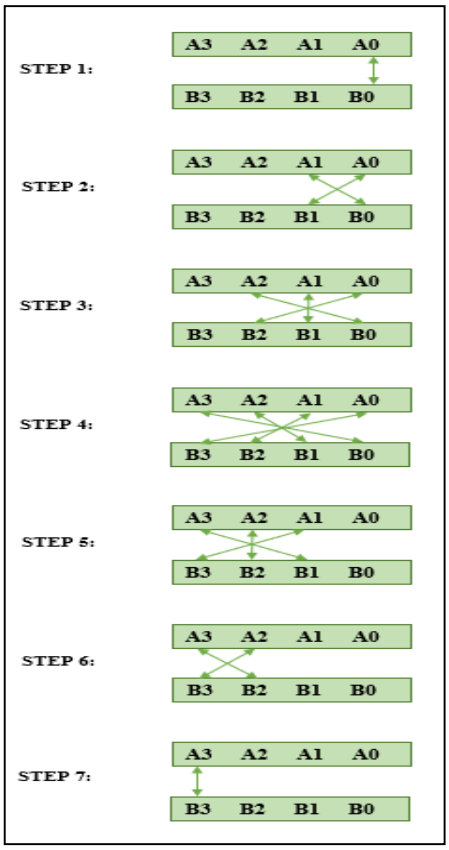

Fig. 5. 4x4 Bit Vedic Multiplication line diagram.

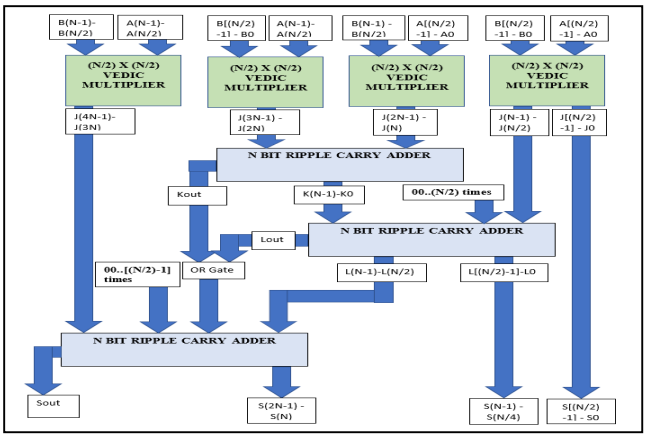

Fig. 6. Proposed N x N-Bit Vedic Multiplier

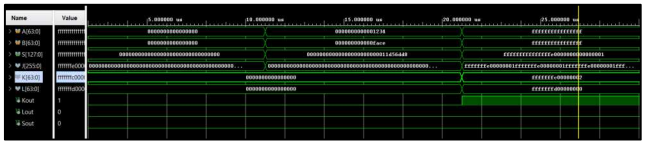

Fig. 7. Simulation results of 64 x 64 bit Vedic Multiplier.

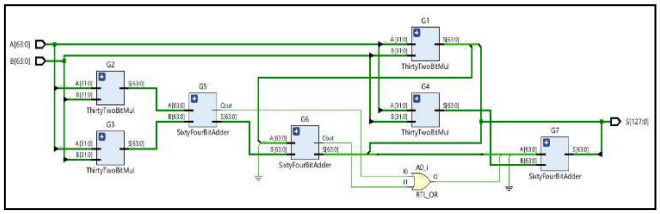

Fig. 8. Schematic of 64 x 64 bit Vedic Multiplier.

The block diagram of the Vedic Multiplier of size N x N-Bit is illustrated in Fig. 6. This design incorporates four Vedic multipliers of size N/2, along with three adders and one OR gate to execute the multiplication process. The adders used here are Ripple Carry Adders.

The simulation outcomes of the Vedic Multiplier of size 64 x 64-Bit are displayed in Fig. 7.

The design of a 64 x 64-Bit Vedic Multiplier involves the utilization of four 32 x 32-Bit Vedic Multipliers, three 64-Bit Adders, and one OR Gate, as illustrated in Fig. 8.

The implementation is done after the process of synthesis of the Verilog code. The synthesis tool maps the synthesized netlist onto the FPGA's configurable logic blocks (CLBs), routing resources, and I/O pins. It also performs placement and routing, deciding where to place the different logic elements within the FPGA and how to connect them. Implementation gives 'Implemented Design' on the device and provides various performance reports. The 'Implemented Design' is automatically placed and routed by Xilinx Vivado. The performance reports can be accessed and analysed from the "Reports" tab.

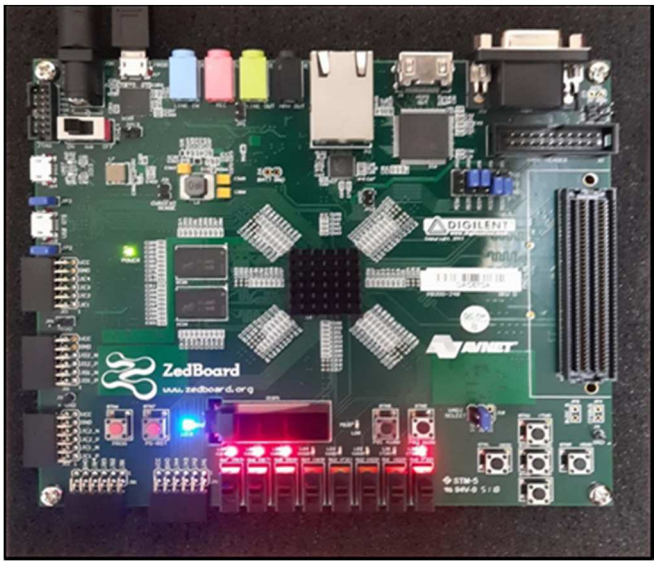

Fig. 9. FPGA Implementation of Vedic Multiplier.

V. Methodology

The performance of the multipliers needs to be accessed and analysed. The prominent parameters of performance we take note of are Maximum Delay Paths, Total On-Chip Power, and Logic Utilization.

The Maximum Path Delay indicates the longest delay among all paths within the circuit and impacts the maximum achievable clock frequency and overall performance. The Total On-Chip Power is composed of both dynamic and static power. Dynamic power seems to be the dominant factor in total power consumption. Logic Block Utilization represents the resource consumption and impacts the overall area efficiency.

The design needs to be implemented on FPGA by uncommenting of used components from the XDC file and assignment of pins to the I/O ports is done before we do the implementation.

The implementation of the Vedic and Booth Multipliers on FPGA is done by using the chosen FPGA's the XDC file as a constraint file. Then, uncomment the used components of the FPGA and assign the switches, LEDs and internal clock to the inputs and outputs. Obtain the implementation afterward. As the implementation is done successfully, the bit stream is generated which is then used to configure the FPGA Board. The FPGA board is then set up for configuration with the required tools and cables. Usually, the FPGA is connected to the USB port of the laptop for configuration.

Now, we load the bitstream onto the FPGA using Vivado Hardware Manager. The 'Open the Target device' option is selected from Hardware Manager. When the FPGA device is configured, the "DONE" Blue LED on the FPGA turns HIGH. After configuring the FPGA, functional testing is done to ensure that the design functions correctly on the physical hardware by sliding the user DIP Switches to enter input and obtaining the output across the user LEDs as shown in Fig. 9 above.

VI. Comparison Reports

A comparative analysis between the multipliers based on the Vedic Multiplication Algorithm and the Booth Algorithm shows that the latter is superior to the former. The parameters used for the comparison are Maximum Path Delays (in ns), Power Utilization (in Watts) and Configurable Logic Blocks (CLB) Utilization.

Table 1 presents a comparison of the maximum path delays (in ns), while Table 2 provides a breakdown of power utilization (in Watts) for both Vedic as well as Booth multiplier algorithms. Table 3 illustrates the comparison of logic block utilization between the FPGA implementations of the Vedic Multiplier and the Booth Multiplier discussed in this paper. Furthermore, Table 4 outlines a comparison of the maximum path delay (in ns) among the 32 x 32-bit Array multiplier [24], 32 x 32-bit Vedic Multiplier, and 32 x 32-bit Booth Multiplier.

TABLE I. Maximum Path Delays (in ns)

S. No.	Maximum Path Delays (in ns)		
	Multipliers	***Vedic Algorithm***	***Booth Algorithm***
1.	4x4 Multiplier	6.570	3.346
2.	8x8 Multiplier	9.114	3.709
3.	16x16 Multiplier	12.800	3.807
4.	32x32 Multiplier	18.413	4.429

S. No.	Maximum Path Delays (in ns)		
	Multipliers	*Vedic Algorithm*	*Booth Algorithm*
5.	64x64 Multiplier	27.897	4.927

TABLE II. POWER UTILIZATION (IN WATTS)

S. No.	Power Utilization (in Watts)		
	Multipliers	*Vedic Algorithm*	*Booth Algorithm*
1.	4x4 Multiplier	3.402	5.636
2.	8x8 Multiplier	11.456	11.046
3.	16x16 Multiplier	57.726	51.347
4.	32x32 Multiplier	154.362	102.033
5.	64x64 Multiplier • Static Power • Dynamic Power • Total Power	 4.403 428.999 433.402	 4.403 205.793 210.195

TABLE III. CONFIGURABLE LOGIC BLOCKS

S. No.	Configurable Logic Blocks		
	Multipliers	*Vedic Algorithm*	*Booth Algorithm*
1.	4x4 Multiplier	34	20
2.	8x8 Multiplier	111	34
3.	16x16 Multiplier	486	77
4.	32x32 Multiplier	1918	151
5.	64x64 Multiplier • CLB LUTs • CLB Registers	 7667 -	 299 200

TABLE IV. DELAY COMPARISON OF DIFFERENT ALGORITHMS (IN NS)

S. No.	Maximum Path Delays for 32x32 Multipliers (in ns)		
	32 Bit Array Multiplier	*32 Bit Vedic Algorithm*	*32 Bit Booth Algorithm*
1.	72.986	18.413	4.429

VII. CONCLUSIONS

The Power Consumption as well as the Logic Block Utilization for both the algorithms seems to have an upward trend as the operand sizes increase. But, as the Maximum Path Delays in the Vedic Multiplication Algorithm show a steady increase, it fairly remains similar in the Booth Algorithm as operands increase in size. In the 64 x 64-bit multiplication scenario, the Booth Algorithm showcases an 82.33% reduction in delay, a 51.50% decrease in power consumption, and a substantial 96.10% decrease in logic utilization compared to the Vedic multiplier in these specific parameters. It is observed that the only parameter where Vedic Multiplier performs better than the Booth Algorithm is for the Power Utilization for the 4 x 4 Bit Multiplier rendering it useful only for small bit multiplications.

The 32 x 32-bit Array multiplier exhibits the highest delay, rendering it the slowest of the three. The 32 x 32-bit Vedic multiplier and Booth multiplier demonstrate reductions in delay by approximately 74.77% and 93.93% respectively, compared to the delay of the 32 x 32-bit Array multiplier.

REFERENCES

[1] A. S. Prabhu, and V. Elakya, “Design of modified low power booth multiplier.” 2012 International Conference on Computing, Communication and Applications, Dindigul, India, 2012, pp. 1-6.

[2] G. Haridas, & D.S. George, “Area efficient Low Power Modified Booth Multiplier for FIR Filter”. International Conference on Emerging Trends in Engineering, Science and Technology, Vol.24, pp. 1163-1169.

[3] Vaidya, S., & Dandekar, D., (2010), “Delay-Performance Comparison of Multipliers in VLSI Circuit Design”. 2010 International Journal of computer Networks & Communications (IJCNC), Vol.2, No.4, July 2010.

[4] Jagadguru Swami Sri Bharati Krishna Trithaji Maharaja, “Vedic Mathematics”, Motilal Banarsidas, Varanasi, India, 1986.

[5] Booth, Andrew Donald, "A Signed Binary Multiplication Technique" (PDF). The Quarterly Journal of Mechanics and Applied Mathematics Oxford University Press. pp. 100–104.

[6] S. Venkatachalam, H. J. Lee, and S.B. Ko, “Power Efficient Approximate Booth Multiplier,” 2018 IEEE International Symposium on Circuits and Systems (ISCAS), Florence, Italy, 2018, pp.1-4.

[7] A. Jais, & P. Palsodkar, “Design and Implementation of 64 bit multiplier using Vedic algorithm.” 2016 International conference on Communication and Signal Processing (ICCSP), Melmaruvathur, India, 2016, pp. 0775-0779.

[8] U. Narula, R. Tripathi, and G. Wakhle, “High speed 16-bit digital Vedic multiplier using FPGA,’2015 2nd International Conference on Computing for Sustainable Global Development (INDIACom), New Delhi, India, 2015, pp.121-124

[9] Sachdeva, Nitin, and Tarun Sachdeva. "An FPGA Based Real-time Histogram Equalization Circuit for Image Enhancement." International Journal of Electronics Communication Technology (IJECT) 1, no. 1 (2010).

[10] Hemangi Patil, S.D. Sawant” Design and Implementation of Energy Efficient Vedic Multiplier using FPGA” 2019 International Conference on Information Processing Vishwakarma Institute of Technology. Dec 18-19, 2019

[11] S. Lad and V. S. Bendre, "Design and Comparison of Multiplier using Vedic Sutras," 2019 5th International Conference On Computing, Communication, Control And Automation (ICCUBEA), IEEE 2019, pp. 1-5.

[12] S. Akhter and S. Chaturvedi, "Modified Binary Multiplier Circuit Based on Vedic Mathematics," 2019 6th IEEE International Conference on Signal Processing and Integrated Networks (SPIN), 2019

[13] A. Bisoyi, M. Baral and M. K. Senapati, "Comparison of a 32-bit Vedic multiplier with a conventional binary multiplier," ICACCCT IEEE 2014.

[14] D. K. Kahar and H. Mehta, "High speed vedic multiplier used vedic mathematics," 2017 International Conference on Intelligent Computing and Control Systems , IEEE 2017, pp. 356-359.

[15] Neeraj Kumar Mishra, Subodh Wairya, “Low power 32x32 bit multiplier architecture based on Vedic Mathematics using Virtex 7 low power design” in IJRREST Vol. 2, Issue-2, June 2013.

[16] “Xilinx ISE 9.2i User Manual”, Xilinx Inc, USA, 2007.

[17] Verilog HDL by Samir Palnitkar.

[18] M.E.Paramasivam, and Dr.R.S.Sabeenian, “An Efficient Bit Reduction Binary Multiplication Algorithm using Vedic Methods”, 2010 IEEE 2nd International Advance Computing Conference.

[19] M. Ramalatha, K. Deena Dayalan, P. Dharani, “High Speed Energy Efficient ALU Design using Vedic Multiplication Techniques”, ACTEA 2009 July 15-17, 2009 Zouk Mosbeh, Lebanon

[20] Neha Goyal, Khushboo Gupta, and Renu Singla, “Study of Combinational and Booth Multiplier”, International Journal of Scientific and Research Publications, Volume 4, Issue 5, pp. 384-388, May 2014

[21] Leonard Gibson Moses S, Thilagar M, “VHDL Implementation of High Performance RC6 Algorithm Using Ancient Indian Vedic Mathematics”, IEEE Xplore.

[22] Asmita Haveliya, “A Novel Design for High Speed Multiplier for Digital Signal Processing Applications (Ancient Indian Vedic mathematics approach)”, International Journal of Technology and Engineering System (IJTES), Vol.2, No.1, Jan-March, 2011.

[23] Nitin Sachdeva and Tarun Sachdeva, “An FPGA Based Real-Time Histogram Equalization Circuit For Image Enhancement,” IJECT Vol. 1, Issue1,December2010.

[24] K. N. Singh and H. Tarunkumar, "A review on various multipliers designs in VLSI," 2015 Annual IEEE India Conference (INDICON), New Delhi, India, 2015, pp. 1-4, doi: 10.1109/INDICON.2015.7443420.